

# שפות תכנות 2025

## תוכן עניינים

הסיכום עובד לפי הפרקים בספר הלימוד באנגלית, ולא עוקב אחר הפרקים בסדר המופיעים במדריך הלמידה. הסיבה היא כי מדריך הלמידה מבלבל את המוח.

|    |       |                                        |
|----|-------|----------------------------------------|
|    | פרק 1 | קבוצות נתונים אינדוקטיביות             |
| 2  | 1.1   | מבנים המאופיינים רקורסיבית             |
| 3  | 1.2   | גזירת תוכניות רקורסיביות               |
|    | פרק 2 | הפשטת נתונים                           |
| 6  | 2.1   | הגדרת נתונים באמצעות ממשקים            |
| 7  | 2.2   | אסטרטגיות ייצוג לטיפוסי נתונים         |
| 10 | 2.3   | ממשקים לטיפוס נתונים רקורסיבי          |
| 11 | 2.4   | כלי להגדרת טיפוסי נתונים רקורסיביים    |
| 12 | 2.5   | ABSTRACT SYNTAX AND ITS REPRESENTATION |
|    | פרק 3 | שפות 'ויהי' ו- 'וישגור'                |
| 14 |       | שפת LET                                |
| 19 |       | שפת PROC                               |
| 20 |       | שפת LETREC                             |
|    | פרק 4 |                                        |
| 23 |       | שפת EXPLICIT-REFS                      |
| 27 |       | שפת IMPLICIT-REFS                      |
| 29 |       | סיכום שפות                             |
|    | פרק 7 | שפות Type-Safe                         |
| 31 |       | שפת Checked                            |
| 31 |       | שפת Inferred                           |
| 32 |       | אלגוריתם Substitutions                 |

## פרק 1 – קבוצות נתונים אינדוקטיביות

### 1.1 מבנים המאופיינים רקורסיבית

בעת כתיבת פרוצדורה, חשוב לדעת אילו ערכים יכולים להתקבל כקלט ואילו ערכים תקינים להחזרה. לעיתים קבוצות הערכים האלו מורכבות, ולכן נדרש אופן פורמלי להגדרתן.

ניתן לאפיין קבוצות או מבנים רקורסיביים בכמה דרכים:

1. הגדרה מלמעלה למטה (Top Down)
2. הגדרה מלמטה למעלה (Bottom-Up)
3. הגדרת כללי גזירה (Rules of inference, Grammar)

ההגרה הכי פשוטה היא בעזרת דקדוק. ניתן פה כמה דוגמאות שונות לפונקציות ומבני נתונים שמוגדרים היטב לפי הדקדוק שלהם.

#### רשימות של מספרים –

בשפת Scheme, list-of-int היא רשימת מספרים אמ"מ הרשימה ריקה או היא זוג כאשר car הוא cdr int הוא רשימה של int.

$\text{List-of-Int} ::= () \mid (\text{Int} . \text{List-of-Int})$

$\text{List-of-Int} ::= (\{\text{Int}\} *)$

Example:

$\Rightarrow (14 . ())$

#### רשימות של סימבולים –

בשפת Scheme, s-list היא רשימה של s-exp, כאשר s-exp הוא סימבול או s-list.

$S\text{-list} ::= (\{S\text{-exp}\} *)$

$S\text{-exp} ::= \text{Symbol} \mid S\text{-list}$

Example:

(a b c)

(an (((s – list)) (with () lots) ((of) nesting)))

#### עץ בינארי -

עץ בינארי עם עלים ממוספרים וצמתים פנימיים מסומנים עם סימבולים.

עץ חיפוש בינארי מוגדר להיות ריק, בעל int, או בעל שני תתי עצים.

$\text{Bintree} ::= \text{Int} \mid (\text{Symbol Bintree Bintree})$

$\text{Binary-search-tree} ::= () \mid (\text{Int Binary-search-tree Binary-search-tree})$

Example:

1

(foo 1 2)

(bar 1 (foo 1 2))

(baz

(bar 1 (foo 1 2))

(biz 4 5))

## ביטוי Lambda –

ביטוי lambda מורכבים מהצהרות של משתנים. משתנים אלו הם פונקציות שלוקחות argument יחיד ומספר פעולות.

$LcExp ::= Identifier$

$::= (lambda (Identifier) LcExp)$

$:= (LcExp LcExp)$

where an identifier is any symbol other than lambda.

בביטוי lambda, המשתנה שמופיע בהגדרה הוא ה-bound variable - כלומר, הוא קושר אליו כל מופע של אותו מזהה בגוף הביטוי.

לדוגמה:

$(lambda (x)(+ x 5))$

בביטוי הזה, x הוא bound variable. זוהי פרוצדורה שמחזירה את הערך של הארגומנט בתוספת 5.

כעת נבחן:

$((lambda (x)(+ x 5)) (- x 7))$

המופע האחרון של x אינו מתייחס ל-x שב-lambda, אלא למשהו חיצוני. זאת כיוון שה-lambda קושר רק את המופעים בתוך הגוף הפנימי שלו.

בהמשך הפרק מוסבר על אינדוקציה מבנית... אבל למדנו את זה כל כך הרבה במסגרת התואר שמיותר שוב...

## 1.2 גזירת תוכניות רקורסיביות

נעשה שימוש בהגדרה אינדוקטיבית כדי לאפיין קבוצות מורכבות. כאשר איבר שייך לקבוצה שמוגדרת אינדוקטיבית, ניתן לנתח אותו לפי הדרך שבה נבנה מאיברים קטנים יותר.

פרוצדורות רקורסיביות מבוססות על העיקרון הבא:

אם נוכל לצמצם בעיה לתת-בעיה קטנה יותר, נוכל לקרוא לאותה פרוצדורה כדי לפתור את תת-הבעיה. הפתרון של תת-הבעיה ישמש לפתרון הבעיה המקורית.

כל קריאה מבצעת הפשטה לבעיה קטנה יותר, עד שבסופו של דבר מגיעים למקרה בסיס — שניתן לפתור אותו ישירות ללא קריאה רקורסיבית נוספת.

### 1.2.1 הפרוצדורה הבסיסית **list-length** מחזירה את כמות האיברים ברשימה.

**list-length:**  $List \rightarrow Int$

**usage:** (list-length *l*) = the length of *l*

```
(define list-length
  (lambda (lst)
    (if (null? lst)
        0
        (+ 1 (list-length (cdr lst))))))
```

### 1.2.2 הפרוצדורה הבסיסית **nth-element** מחזירה את האיבור במקום ה-*n* במרשימה.

**nth-element:**  $List \times Int \rightarrow SchemeVal$

**usage:** (nth-element *lst n*) = the *n*-th element of *lst*

```
(define nth-element
  (lambda (lst n)
    (if (null? lst)
        (report-list-too-short n)
        (if (zero? n)
            (car lst)
            (nth-element (cdr lst) (- n 1))))))
```

```
(define report-list-too-short
  (lambda (n)
    (eopl:error 'nth-element
      "List too short by ~s elements.~%" (+ n 1))))
```

### 1.2.3 הפרוצדורה **remove-first** מוחקת את המופע הראשון של סימבול נתון כלשהו.

**remove-first:**  $Sym \times Listof(Sym) \rightarrow Listof(Sym)$

```
(define remove-first
  (lambda (s los)
    (if (null? los)
        '()
        (if (eqv? (car los) s)
            (cdr los)
            (cons (car los) (remove-first s (cdr los)))))))
```

### 1.2.4 הפרוצדורה **occurs-free** בודקת אם משתנה כלשהו הוא חופשי בביטוי כלשהו.

**occurs-free?:**  $Sym \times LcExp \rightarrow Bool$

**usage:** returns #t if the symbol var occurs free in exp, otherwise returns #f.

```

(define occurs-free?
  (lambda (var exp)
    (cond
      ((symbol? exp) (eqv? var exp))
      ((eqv? (car exp) 'lambda)
       (and
        (not (eqv? var (car (cadr exp))))
        (occurs-free? var (caddr exp))))
      (else
       (or
        (occurs-free? var (car exp))
        (occurs-free? var (cadr exp)))))))

```

**1.2.5** הפרוצדורה **subst** מקבלת 3 ארגומנטים, תו ישן, חדש, ורשימה, ומחליפה את כל התווים הישנים בחדשים.

**subst:**  $\text{Sym} \times \text{Sym} \times \text{S-list} \rightarrow \text{S-list}$

```

(define subst
  (lambda (new old slist)
    (if (null? slist)
        '()
        (cons
         (subst-in-s-exp new old (car slist))
         (subst new old (cdr slist))))))

```

**subst-in-s-exp:**  $\text{Sym} \times \text{Sym} \times \text{S-exp} \rightarrow \text{S-exp}$

```

(define subst-in-s-exp
  (lambda (new old sexp)
    (if (symbol? sexp)
        (if (eqv? sexp old) new sexp)
        (subst new old sexp))))

```

רעיון Follow – The – Grammar

בינתיים הצלחנו לפתח שיטה כאשר אנחנו מגדירים פונקציה / מתודה שאמורה לפעול על מבנה נתונים שמוגדר אינדוקטיבית (רקורסיה / דקדוק).

המבנה של הפונקציה / מתודה צריך לעקוב אחר המבנה הדקדוקי של מבנה הנתונים.

- כותבים פונקציה אחת עבור כל non-terminal בדקדוק. הפונקציה תהיה אחראית על טיפול במידע הקשור לnon-terminal זה, וזה בלבד.
- בכל פונקציה, כתוב אלטרנטיבה אחת עבור כל כלל גזירה אפשרי של אותו non-terminal. עבור כל non-terminal שמופיע בצד ימין של כלל הגזירה, קרא בצורה רקורסיבית לפונקציה של אותו non-terminal.

## פרק 2 – הפשטת נתונים

הפשטת נתונים (Data Abstraction) מאפשרת לבדוד בין מפרט (Specification) השימוש לבין פרטי המימוש (Implementation). המפרט של טיפוס הנתונים מתאר את מהות הנתונים, את הפעולות המוגדרות עליהם ואת תכונותיהם. לעומת זאת, המימוש קובע את אופן ייצוג הנתונים ומפסק את הקוד הפועל על הייצוג הנבחר.

### 2.1 הגדרת נתונים באמצעות ממשקים

כאשר בוחרים לייצג קבוצת ערכים בצורה מסוימת, יוצרים טיפוס נתונים חדש: כזה שערכיו הם הייצוגים, והפעולות עליו הן הפונקציות שפועלות עליהם.

הייצוג עשוי להיות מורכב, ולכן כדאי להסתיר את פרטי המימוש באמצעות הפשטת נתונים. כך ניתן גם להחליף את הייצוג בעתיד מבלי לשנות את כל הקוד.

הפשטת נתונים מחלקת טיפוס נתונים לשני חלקים:

**Interface** – מתאר מהו הטיפוס, אילו פעולות קיימות עליו, ואילו תכונות הן קבועות.

**Implementation** – מספק את הייצוג בפועל ואת הקוד שמממש את הפעולות על הייצוג הזה.

כאשר שאר הקוד משתמש בטיפוס הנתונים רק דרך הממשק, הוא representation-independent, כלומר, בלתי תלוי בייצוג הפנימי.

### דוגמה לממשק: Natural Numbers

הנתונים הם מספרים ממשיים, והממשק מכיל 4 פרוצדורות:

$(\text{zero}) = 0$

$(\text{is-zero? } n) = \#t \quad \text{אם } n = 0$

$\#f \quad \text{אם } n \neq 0$

$(\text{successor } n) = n + 1 \quad (n \geq 0)$

$(\text{predecessor } (n + 1)) = n \quad (n \geq 0)$

המפרט לא מחייב ייצוג מסוים של המספרים — רק שהפעולות יעבדו בצורה עקבית עם ההגדרות. בספר ישנם 3 דוגמאות לייצוגים שונים של מספרים, אשר כולם מממשים את הממשק הנ"ל (ייצוג אונרי, ייצוג בשפת Scheme ו-Big Num כלשהו שאינו 10).

כעת, אפשר לעשות שימוש בממשק על מנת לחשב את פעולת החיבור בין שני מספרים:

```
(define plus
  (lambda (x y)
    (if (is-zero? x)
        y
        (successor (plus (predecessor x) y)))))
```

בממשק לרוב יהיו **בנאים ומתבוננים** :

**Constructors** – בונים ערכים של הטיפוס (zero, successor, predecessor)

**Observers** – בודקים או מחלצים מידע מהערכים (is-zero)

## 2.2 אסטרטגיות ייצוג לטיפוסי נתונים REPRESENTATION STRATEGIES

כאשר משתמשים בהפשטת נתונים, התוכניות הופכות להיות בלתי תלויות בייצוג. כלומר, ניתן לשנות את הייצוג של טיפוס הנתונים בלי לשנות את הקוד שמשתמש בו - רק את מימוש הפונקציות שבממשק. תכונה זו מנוצלת רבות בהמשך.

### 2.2.1 ממשק של Environment

Environment הוא פונקציה שסביבתה היא קבוצת משתנים סופית, וטווחה הוא קבוצת כל הערכים האפשריים ב-Scheme. לפי מוסכמה מתמטית, פונקציה סופית מיוצגת כקבוצה סופית של זוגות סדורים :

$$\{(var_1, val_1), \dots, (var_n, val_n)\}$$

כאשר כל  $var_i$  הוא משתנה שונה, ו- $val_i$  הוא כל ערך חוקי ב-Scheme. הערך המשוך למשתנה בתוך סביבה נקרא binding של המשתנה.

ממשק Environment כולל שלוש פרוצדורות :

$$(empty - env) = \emptyset$$

$$(extend - env \ var \ v \ f) = g$$

$$(apply - env \ f \ var) = f(var)$$

$$g(var_1) \begin{cases} v & \text{if } var_1 = var \\ f(var_1) & \text{otherwise} \end{cases} \quad \text{כאשר } g(var_1) \text{ מוגדר כך :}$$

empty-env – Constructor, מייצרת את הסביבה הריקה.

extend-env – Constructor, יוצרת סביבה חדשה על בסיס קיימת, עם binding חדש או מחודש למשתנה.

apply-env – Observer, מפעילה סביבה על משתנה ומחזירה את ערכו. דוגמה :

```
(define e
  (extend-env 'd 6
    (extend-env 'y 8
      (extend-env 'x 7
        (extend-env 'y 14
          (empty-env)))))))
```

והסביבה **e** שנוצרה תכיל  $e(d) = 6$   $e(x) = 7$   $e(y) = 8$

## 2.2.2 ייצוג של Environment (מבני נתונים)

נבחין כי כל Environment ניתן לבנייה ע"י אתחול הסביבה הריקה, ולאחר מכן ביצוע פעולות של extend-env לפחות n פעמים, עבור  $n \geq 0$ . מכאן, שניתן לייצג זאת בתור דקדוק:

$Env - exp ::= (empty - env) | (extend - env Identifier Scheme\_Value Env - exp)$

ייצוג זה מאפשר לתאר את הסביבה באמצעות מבני נתונים רקורסיביים — ממש כמו עץ או רשימה.

### Interpreter Recipe

1. נתבונן בפיסת מידע.
2. נחליט איזה סוג Data הוא מייצג.
3. נחלץ את ה-Components של המידע, ונבצע את הפעולות הנכונות איתם.

$Env = (empty-env) | (extend-env Var SchemeVal Env)$

$Var = Sym$

**empty-env:**  $() \rightarrow Env$

```
(define empty-env
  (lambda () (list 'empty-env)))
```

**extend-env:**  $Var \times SchemeVal \times Env \rightarrow Env$

```
(define extend-env
  (lambda (var val env)
    (list 'extend-env var val env)))
```

**apply-env:**  $Env \times Var \rightarrow SchemeVal$

```
(define apply-env
  (lambda (env search-var)
    (cond
      ((eqv? (car env) 'empty-env)
       (report-no-binding-found search-var))
      ((eqv? (car env) 'extend-env)
       (let ((saved-var (cadr env))
             (saved-val (caddr env))
             (saved-env (caddr env)))
         (if (eqv? search-var saved-var)
             saved-val
             (apply-env saved-env search-var))))))
```



```

      (else
        (report-invalid-env env))))))

(define report-no-binding-found
  (lambda (search-var)
    (eopl:error 'apply-env "No binding for ~s" search-var)))

(define report-invalid-env
  (lambda (env)
    (eopl:error 'apply-env "Bad environment: ~s" env)))

```

### 2.2.3 ייצוג פרוצדורלי Procedural Representation

לממשק environment יש תכונה חשובה: יש לו observer יחיד – apply-env. לכן ניתן לייצג סביבה כפרוצדורה שלוקחת משתנה ומחזירה את הערך המשוך לו.

במימוש הזה:

empty-env מחזירה פרוצדורה שמדווחת שגיאה עבור כל משתנה.

extend-env מחזירה פרוצדורה שמכילה את ה-binding החדש, וממשיכה את החיפוש ב saved-env אם המשתנה אינו תואם.

זהו **ייצוג שבו הנתון עצמו** הוא הפעולה שנעשית עליו באמצעות apply-env.

```

(define empty-env
  (lambda ()
    (lambda (search-var)
      (report-no-binding-found search-var))))

(define extend-env
  (lambda (saved-var saved-val saved-env)
    (lambda (search-var)
      (if (eqv? search-var saved-var)
          saved-val
          (apply-env saved-env search-var)))))

(define apply-env
  (lambda (env search-var)
    (env search-var)))

```

מתכון לייצוג פרוצדורלי :

1. זהה ביטויי lambda בקוד שמחזירים ערכים מהטיפוס.

צור עבורם constructors לפי המשתנים החופשיים בהם.

2. זהה את כל המקומות בקוד (כולל בגוף ה-constructors) שבהם מופעל ערך מהטיפוס. החלף אותם בקריאות ל-apply המתאימה.

כך יוצרים ממשק שמכיל את כל ה-Constructors, ואת Apply, והקוד של הלקוח יהיה בלתי תלוי ייצוג.

## 2.3 ממשקים לטיפוס נתונים רקורסיבי INTERFACES FOR RECURSIVE DATA TYPES

בפרק 1 עסקנו רבות בטיפוסי נתונים רקורסיביים, כולל Lc-exp — טיפוס לביטויי  $\lambda$ -calculus (ביטוי למבדה)

הפרוצדורה occurs-free? שראינו שם לא הייתה קריאה במיוחד, כי היא השתמשה בגישה ישירה ל-car. ללא הסתרת הייצוג.

**פתרון :** ממשק חדש לביטויי  $\lambda$ -calculus

ממשק זה ישפר קריאות ויאפשר ניתוק בין מימוש וייצוג. הממשק יכיל Constructors ושני סוגים של Observers, שמם הוא Predicates ו-Extractors.

**Constructors יהיו:**

|                    |                                                                |
|--------------------|----------------------------------------------------------------|
| <b>var-exp:</b>    | $\text{Var} \rightarrow \text{Lc-exp}$                         |
| <b>lambda-exp:</b> | $\text{Var} \times \text{Lc-exp} \rightarrow \text{Lc-exp}$    |
| <b>app-exp:</b>    | $\text{Lc-exp} \times \text{Lc-exp} \rightarrow \text{Lc-exp}$ |

**Predicates (מנבאים) יהיו:**

|                     |                                         |
|---------------------|-----------------------------------------|
| <b>var-exp?:</b>    | $\text{Lc-exp} \rightarrow \text{Bool}$ |
| <b>lambda-exp?:</b> | $\text{Lc-exp} \rightarrow \text{Bool}$ |
| <b>app-exp?:</b>    | $\text{Lc-exp} \rightarrow \text{Bool}$ |

**Extractors (מחלצים) יהיו:**

|                                  |                                        |
|----------------------------------|----------------------------------------|
| <b>var-exp-&gt;var:</b>          | $\text{Lc-exp} \rightarrow \text{Var}$ |
| <b>lambda-exp-&gt;bound-var:</b> | $\text{Lc-exp} \rightarrow \text{Var}$ |

**lambda-exp->body:** Lc-exp  $\rightarrow$  Lc-exp

**app-exp->rator:** Lc-exp  $\rightarrow$  Lc-exp

**app-exp->rand:** Lc-exp  $\rightarrow$  Lc-exp

כל אחד מהExtractors ייחלץ את החלק הרלוונטי של הביטוי למבדה. כעת אפשר לכתוב גרסה יותר קריאה עבור occurs-free?, שתלוי אך ורק בממשק. מימוש זה תלוי רק בממשק, ולכן מתאים לכל ייצוג פנימי שמתממשק אליו. ניתן לראות את המימושים השונים בספר / קבצים של פרק 2.

(define occurs-free?

(lambda (search-var exp)

(cond

((var-exp? exp) (eqv? search-var (var-exp->var exp)))

((lambda-exp? exp)

(and

(not (eqv? search-var (lambda-exp->bound-var exp)))

(occurs-free? search-var (lambda-exp->body exp))))

(else

(or

(occurs-free? search-var (app-exp->rator exp))

(occurs-free? search-var (app-exp->rand exp))))))

תכנון ממשק עבור טיפוס נתונים רקורסיבי

1. צור constructor אחד עבור כל סוג של נתון בטיפוס.
2. צור predicate אחד לזיהוי כל סוג כזה (עבור כל constructor)
3. צור extractor אחד לכל רכיב שמועבר ל-constructor.

## 2.4 כלי להגדרת טיפוס נתונים רקורסיביים

עבור טיפוס נתונים מורכבים, תהליך כתיבת ממשק לפי המתכון עלול להפוך למעייף. לשם כך מוצג כאן כלי אוטומטי בScheme, ששמו define-datatype, והוא מאפשר להגדיר טיפוסים רקורסיביים בצורה נוחה, ולכתוב ממשק עבורם בצורה עקבית.

(define-datatype lc-exp lc-exp?

(var-exp

(var identifier?))

(lambda-exp

(bound-var identifier?)

(body lc-exp?))

```
(app-exp
  (rator lc-exp?)
  (rand lc-exp?)))
```

שלושת ה-constructors הם: var-exp, lambda-exp, app-exp

הטיפוס כולו נבדק באמצעות lc-exp?

כל שדה נבדק לפי הפרדיקט שהוגדר לו (למשל: lc-exp? identifier?)

בזכות בדיקות אלו, מובטח שכל ביטוי שנבנה דרך הממשק הוא תקין — אין צורך לבדוק תקינות בזמן ריצה.

במקום להשתמש בפרדיקטורים ואקסטרקטורים, אפשר להשתמש ב-cases על מנת לגלות את סוג ה-Data Type עבורו ה-Data משתייך, ולחלץ את מרכיביו. על מנת להמחיש זאת, נדגים שוב את occurs-free? אך הפעם נשתמש ב-lc-exp.

```
(define occurs-free?
  (lambda (search-var exp)
    (cases lc-exp exp
      (var-exp (var)
        (eqv? var search-var))
      (lambda-exp (bound-var body)
        (and (not (eqv? search-var bound-var))
              (occurs-free? search-var body)))
      (app-exp (rator rand)
        (or (occurs-free? search-var rator)
            (occurs-free? search-var rand))))))
```

כעת ניתן להגדיר את ההצהרה של **define-datatype**:

```
(define-datatype type-name type-predicate-name
  (variant-name
    (field1 predicate1)
    (fieldN predicateN))
  ...)
```

type-name – שם טיפוס הנתונים.

type-predicate-name – פרדיקט שבדק אם ערך שייך לטיפוס.

variant-name מייצג תת-סוג של הטיפוס, עם שדות ושמות שדות משלו.

### כללים לשמות והבדלים

שמות טיפוסים לא יכולים להיות זהים.

שמות variants חייבים להיות ייחודיים – גם אם שייכים לטיפוסים שונים.

שם של טיפוס לא יכול לשמש כ-variant.

### מה נוצר אוטומטית?

עבור כל variant נוצר constructor עם אותו שם. לדוגמה, עבור :

```
(define-datatype lc-exp lc-exp?
```

```
(var-exp (var identifier?)))
```

נוצר :

```
(var-exp 'x)
```

ותפקידו ליצור data values ששייכים לאותו variant.

כל constructor בודק את הטיעונים שלו מול הפרדיקטים המתאימים, ושומר אותם במבנה הנתונים שנוצר.

## **ABSTRACT SYNTAX AND ITS REPRESENTATION 2.5**

לא החלטתי אם זה פרק רלוונטי או לא...נראלי פשוט תרגיל 4.45 וההסבר של הפתרון במדריך מכסה...  
אם רוצים סקירה יותר מקיפה אפשר להסתכל בהרצאה 5 של דני כלפון מתחילת ההרצאה ועד דקה 44:00.

## פרק 3 – שפות 'ויהי' ו- 'וישגור'

### LET LANGUAGE SYNTAX 3.1

Program ::= Expression

a-program (exp1)

Expression ::= Number

const-exp (num)

Expression ::= -(Expression , Expression)

diff-exp (exp1 exp2)

Expression ::= zero? (Expression)

zero?-exp (exp1)

Expression ::= if Expression then Expression else Expression

if-exp (exp1 exp2 exp3)

Expression ::= Identifier

var-exp (var)

Expression ::= let Identifier = Expression in Expression

let-exp (var exp1 body)

שפת LET היא שפה פשוטה מאוד. התחביר של השפה כולל ביטויים בלבד, כלומר כל תוכנית היא ביטוי. ביטוי יכול להיות אחד מהבאים: קבוע מספרי, פעולת חיסור, בדיקת אפס (zero-test), תנאי (if), משתנה, או ביטוי .let

לדוגמה הביטוי: ((scan&parse "-(55, -(x,11))") (scan&parse) זו פונקציה של SLLGen.

מומר לתחביר מופשט כך:

```
 #(struct:a-program
  #(struct:diff-exp
    #(struct:const-exp 55)
    #(struct:diff-exp
      #(struct:var-exp x)
      #(struct:const-exp 11))))
```

## LET SPECIFICATION 3.2

נתבונן בקבוצה המקור של שפת LET ונשייך את הפעולות בקבצים.

### קובץ lang.scm

בקובץ זה נמצאים שני המרכיבים העיקריים לבניית שפה.

#### 1. Lexical Representations – the-lexical-spec

חלק זה מגדיר איך טקסט נע (raw text) מחולק ל-tokens מתאימים (כמו עבודת Lexer).  
הTokens העלו נשלחים לאחר מכן לParser והוא עושה בהם שימוש.

; Lexical representation according to SLLgen

(define the-lexical-spec

'((whitespace (whitespace) skip)

(comment ("% (arbno (not #¥newline)))) skip)

(identifier

(letter (arbno (or letter digit "\_ "-" "?")))

symbol)

(number (digit (arbno digit)) number)

(number ("-" digit (arbno digit)) number)

(name ("@" letter (arbno letter)) string)

; "@" with one or more letters afterwords. Type returned: string

))

#### 2. Grammar Rules – the-grammar

מגדיר את התחביר של השפה בעזרת ה-Tokens מהסעיף הקודם. כל משפט בשפה נהפך לסוג מסויים של משפט, ויש לו פונקציה שמפעילה אותו / מחשבת אותו.

אצלינו זה יוצא שכמעט כל המשפטים הם מסוג expression, אך כל משפט מופעל על ידי פונקציה אחרת. Zero?-exp, mul-exp וכו'.

פונקציות אלו יושבות בקובץ המפרש, אשר מפרש את המשפט ומחזיר ביטוי עבורו.

פונקציה זו היא בעצם ה-Parser שלנו, והיא יוצרת את ה-AST שמורכב מביטויים שונים. שלב הבא יהיה לעבור על ה-AST ולחשב את ערכיו.

### קובץ interp.scm

The file interp.scm is the interpreter for the let language. It evaluates parsed expressions based on the language's semantics, using list pattern matching and an environment model.

There are two main functions defined in this file:

1. The first is `value-of-program`. This function takes a single argument `pgm`, which is a parsed program (`pgm` is a root of an AST Tree, the nodes are all parsed expressions). It simply calls `value-of` on the program, passing it an empty environment. Its purpose is to initiate the evaluation process.
2. The core function is `value-of`. This is a recursive function that interprets any expression in a given environment. It works as follows:
  - If the expression `exp` is a number (checked with `(number? exp)`), it returns the number itself.
  - If the expression is a symbol (a variable), it looks up its value in the environment using `apply-env`.
  - If the expression is a list, it extracts the first element (operator or tag) with `(car exp)` and handles the rest based on its form:
    - If the operator is `'+'`, `'-'`, or `'*'`, it recursively evaluates the two arguments with `value-of`, and applies the appropriate arithmetic operation to the results.
    - If the operator is `'if'`, it evaluates the test expression. If the result is zero, it evaluates the else branch; otherwise, it evaluates the then branch.
    - If the operator is `'let'`, it assumes the expression is of the form `(let (var rhs) body)`. It evaluates the right-hand side (`rhs`), extends the environment with the new variable binding (`var = rhs`), and then evaluates the body in that extended environment.

The entire interpreter is based on list-structured expressions (no custom AST node types), and it relies on helper functions like `apply-env` and `extend-env` for managing variable bindings.



The file `data-structures.scm` defines the core data structure for representing environments, which are used to store variable bindings during evaluation. All environments are represented as tagged lists that begin with the symbol `'environment`.

There are several key parts in this file:

The constructors are `make-empty-env` and `make-env`.

- `make-empty-env` creates an environment with no bindings.
- `make-env` creates an environment given a list of variable names and a matching list of values.

Both constructors wrap the data in a list tagged with `'environment`.

The predicate in this file is `environment?` which checks whether a given object is a tagged list starting with `'environment`.

It uses a helper called `tagged-list?` to perform this check. The helper works by checking that the input is a pair and that its first element matches the expected tag.

The extractors are `env-vars` and `env-vals`.

- `env-vars` returns the list of variable names stored in an environment.
- `env-vals` returns the corresponding list of values.

Both functions assume the input is a valid environment structure.

In summary, the file includes two constructors (`make-empty-env`, `make-env`), one predicate (`environment?`), two extractors (`env-vars`, `env-vals`), and one internal helper (`tagged-list?`). Together, they provide a minimal and consistent way to create, validate, and access environment structures.

## קובץ environments.scm

The file environments.scm defines the operations for working with environments, which are used to map variable names to their values during interpretation. It builds on the environment data structures defined in data-structures.scm.

There are two main operations implemented in this file:

- The first is empty-env, which is a constant that refers to the result of calling make-empty-env. This represents an environment with no variable bindings.
- The second is extend-env, which takes a variable name, a value, and an existing environment. It uses make-env to construct a new environment where the variable is added to the front of the variable list, and the value is added to the front of the value list. This creates a new scope layered over the existing one.

The most important function in this file is apply-env, which takes an environment and a variable name and returns the value bound to that name. It does this by checking if the environment is empty. If it is, it raises an error saying the variable is not bound. If not, it checks whether the variable matches the first name in the variable list. If it does, it returns the corresponding value. Otherwise, it recursively calls apply-env on the rest of the environment.

The file assumes that environments are constructed using the constructors from data-structures.scm, and that variable and value lists are aligned in parallel.

To summarize: this file defines empty-env as the base environment, extend-env to add a new variable binding, and apply-env to look up the value of a variable. It provides the core functionality for variable scope management in the interpreter.

## קובץ top.scm

קובץ שמריץ את התוכנית הרצויה. משתמשים ב-(run "<let lang code>") בשביל להפעיל את הקוד שרוצים בשפת Let. אם רוצים להריץ את כל הקובץ טסטים משתמשים ב-(run-all), ואם רוצים אחד ספציפי אז (run-<test name here>).

## PROC LANGUAGE 3.3

שפה זו מוסיפה פעולות על בסיס שפת LET שמתוארת למעלה.

הפעולות המתווספות הם האפשרות להגדיר פונקציות ולהפעיל אותן עם ערכים ספציפיים לאחר מכן. כמו כן מתווספות עוד תוספות לכל הקבצים השונים על מנת לבצע את ההתאמות הנדרשות.

What changes when going from LET to PROC:

### 1. Grammar Extension (lang.scm)

- New grammar rules added:
  - proc-exp: for anonymous procedure creation  
(expression  
("proc" "(" identifier ")" expression)  
proc-exp)
  - call-exp: for applying a procedure to an argument  
(expression  
("(" expression expression ")")  
call-exp)

### 2. Value Representation (data-structures.scm)

- New variant in define-datatype expval: proc-val, wrapping a proc  
(proc-val  
 (proc proc?)))
- Extractor: expval->proc safely unwraps the proc-val  
;; expval->proc : ExpVal -> Proc  
(define expval->proc  
 (lambda (v)  
 (cases expval v  
 (proc-val (proc) proc)  
 (else (expval-extractor-error 'proc v))))))
- New type proc: stores (var body env) (i.e., a closure)  
;; proc? : SchemeVal -> Bool  
;; procedure : Var X Exp X Env -> Proc  
(define-datatype proc proc?  
 (procedure  
 (var symbol?)  
 (body expression?)  
 (env environment?)))
- Predicate: proc? used in type definitions and checks  
proc? is auto-generated by define-datatype and used to identify values of type proc.

### 3. Interpreter Logic (interp.scm)

- In value-of:
  - proc-exp returns a closure (procedure var body env)  
(proc-exp (var body)  
  (proc-val (procedure var body env)))
  - call-exp extracts the procedure, evaluates the argument, and applies  
(call-exp (rator rand)  
  (let ((proc (expval->proc (value-of rator env)))  
        (arg (value-of rand env)))  
    (apply-procedure proc arg)))
- apply-procedure:
  - Extends the closure environment with the argument value
  - Evaluates the body in this extended environment

```
;; apply-procedure : Proc * ExpVal -> ExpVal
;; Page: 79
(define apply-procedure
  (lambda (proc val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body (extend-env var val saved-env))))))
```

## LETREC LANGUAGE 3.4

שפת LETREC מרחיבה את שפת PROC בכך שהיא מאפשרת קריאות רקורסיביות לפונקציות. זאת אומרת שפונקציה יכולה לקרוא לעצמה.

What changes when going from PROC to LETREC:

### 1. Grammar Extension (lang.scm)

- New grammar rule added:

```
(expression
  ("letrec"
   identifier "(" identifier ")" "=" expression
   "in" expression)
  letrec-exp)
```

### 2. Interpreter Logic (interp.scm)

- In value-of, we add a clause for letrec-exp:  
;Returns the value of the letrec body expression - as a result of extend-env-rec  
;with the correct variables

```
(letrec-exp (p-name b-var p-body letrec-body)
  (value-of letrec-body
    (extend-env-rec p-name b-var p-body env)))
```

Explanation:

- p-name is the name of the recursive procedure (e.g., fact)
- b-var is its parameter (e.g., n)
- p-body is the body of the function (e.g., (if (= n 0) ...))
- letrec-body is the expression in which the function is used (e.g., (fact 5))

This expression evaluates letrec-body in an environment extended with a recursive binding, created by extend-env-rec.

### 3. Environment Structure and Logic

- **Changes in data-structures.scm:**

We add a new variant to the environment datatype:

;Page 86

```
(define-datatype environment environment?
  (empty-env)
  (extend-env
    (bvar symbol?)
    (bval expval?)
    (saved-env environment?)))
```

;; we add a new constructor for a new type of environment, extend-env-rec

;; it receives 4 variables. id, bound var, body expression, and a saved environment (snapshot)

```
(extend-env-rec
  (id symbol?)           ; name of the function (e.g., 'fact)
  (bvar symbol?)         ; formal parameter (e.g., 'n)
  (body expression?)     ; function body
  (saved-env environment?)) ; snapshot of env at time of binding
```

- **Changes in apply-env (inside environments.scm):**

We modify apply-env to handle recursive bindings:

;; Page: 86

```
(define apply-env
  (lambda (env search-sym)
    (cases environment env
      (empty-env ()
        (eopl:error 'apply-env "No binding for ~s" search-sym))
      (extend-env (var val saved-env)
        (if (eqv? search-sym var)
            val
            (apply-env saved-env search-sym)))))
```

```

    (apply-env saved-env search-sym)))
; We add an observer for extend-env-rec
; receives the 4 variables, returns the procedure if the procedure name is found
; else it looks in the saved environment (the snapshot)
(extend-env-rec (p-name b-var p-body saved-env)
  (if (eqv? search-sym p-name)
    (proc-val (procedure b-var p-body env))
    (apply-env saved-env search-sym))))))

```

Explanation:

- If search-sym matches the recursive function name (p-name), we return a new proc-val.
- That procedure captures:
  - o The function's argument (b-var)
  - o Its body (p-body)
  - o And the full current environment (env) — this is what enables recursion.
- If it doesn't match, we recurse into the saved environment (saved-env).

## פרק 4 – שפות עם REFERENCES

בכל השפות בפרק 4, ועל אף העובדה שיש כאן עבודה עם references, עדיין פרמטרים המועברים לפרוצדורה עוברים בצורת call-by-value. בסוף פרק 4 ישנם שתי שפות המדגימות אופנים שונים להעברת פרמטרים לפרוצדורה. האחת היא שפה המדגימה העברת פרמטרים ע"י call-by-reference. השפה השנייה call-by-name (וגם call-by-need) שבה הפרמטרים של פרוצדורה אינם מחושבים ברגע הקריאה לפרוצדורה אלא רק בזמן שהפרוצדורה משתמשת בהם בפועל.

בשפת "ויפנה" (Explicit-Refs) כל העבודה עם references (יצירה, פניה והשמה) מוטלת על המתכנת עצמו.

שפת "וירמוז" (Implicit Refs) מציעה דרך אחרת להתנהלות עם מצביעים ביחס לשפת "ויפנה". החידוש בשפה זו הוא שהשפה דואגת בעצמה ליצירה והפניה אל reference. מבחינת המתכנת בשפת "וירמוז", references הם שקופים, ומעתה הוא אינו צריך ליצור בעצמו reference או לבקש לפנות אליו. יצירת reference נעשית אוטומטית ע"י המפרש בכל פעם שנדרשת כריכה (Binding) למשתנה. בשפה זו ערכים שניתנים לביטוי (expressed values) אינם זהים לערכים שניתנים לציון (Denoted Values).

שפת "ויבגוד" (Mutable-Pairs), שפה זו היא מאפשרת עבודה עם זוגות. הזוגות מכילים references בתור איברים. וניתן לשנות את ערכם. בפרק 3 בספר הלימוד ישנו תרגיל (תרגיל 3.9) שבו נדרש להוסיף לשפת "וייהי" עבודה עם זוגות. ערכי האיברים בזוגות אלה אינם ניתנים לשינוי (immutable). שפת "ויבגוד" בסה"כ מדגימה מימוש של עבודה עם זוגות שאיבריהן ניתנים לשינוי (בזכות העבודה עם references והיכולות לבצע אפקט של השמה).

## EXPLICIT-REFS LANGUAGE 4.2

שפה זו נבנית על שלושת השפות למעלה. שפה זו מאפשרת יצירה וחישוב של מצביעים (references). ההפניות נשמרות במבנה נתונים שנקרא store. בגדול, זוהי רשימה של "תאים" ובכל "תא" יש ערך. נבחין כי שפה זו מאפשרת לעשות הפניה להפניה אחרת, (pointer to pointer to pointer....) מה שהשפה הבאה לא תאפשר. נעבור כעת על התוספות העקריות לשפה בקבצים השונים.

### 1. Grammar Extension (lang.scm)

- New expressions added:

```
(expression
  ("begin" expression (arbno ";" expression) "end")
  begin-exp)

(expression
  ("newref" "(" expression ")")
  newref-exp)

(expression
  ("deref" "(" expression ")")
  deref-exp)

(expression
  ("setref" "(" expression "," expression ")")
  setref-exp)
```

### 2. Interpreter Changes (interp.scm)

- Store Initialization:

```
(define value-of-program
  (lambda (pgm)
    (initialize-store!) ;; Initializes the store data structure.
    (cases program pgm
      (a-program (exp1)
        (value-of exp1 (init-env))))))
```

- New expression handlers in value-of:

```
;; Evaluates a sequence of expressions; returns the final result
(begin-exp (exp1 exps)
  (letrec ((value-of-begins
    (lambda (e1 es)
      (let ((v1 (value-of e1 env)))
        (if (null? es)
            v1
```

```

        (value-of-begins (car es) (cdr es))))))
(value-of-begins exp1 exps)))

;; Allocates a reference in the store and returns its address wrapped in ref-val
(newref-exp (exp1)
  (let ((v1 (value-of exp1 env)))
    (ref-val (newref v1))))

;; Retrieves the value stored at a reference
(deref-exp (exp1)
  (let ((v1 (value-of exp1 env)))
    (let ((ref1 (expval->ref v1)))
      (deref ref1))))

;; Sets the value at a reference to the new evaluated value; returns dummy value
(setref-exp (exp1 exp2)
  (let ((ref (expval->ref (value-of exp1 env))))
    (let ((v2 (value-of exp2 env)))
      (begin
        (setref! ref v2)
        (num-val 23))))      ;dummy result

```

### 3. Value Representation (data-structures.scm)

- Add new variant to expval (new type of possible value returned):

```

(ref-val
  (ref reference?))

```

- Extractor for references:

```
;; expval->ref : ExpVal -> Ref
```

```
;; Retrieves the reference value from a ref-val, errors otherwise
```

```

(define expval->ref
  (lambda (v)
    (cases expval v
      (ref-val (ref) ref)
      (else (expval-extractor-error 'reference v)))))

```



## 4. Store Management (store.scm)

The store is a global list (the-store) representing memory. Each reference is an index into that list.

- Store Initialization

```
;; the-store : global variable holding the memory
(define the-store 'uninitialized)

;; empty-store : () -> Store
;; Returns an empty store
(define empty-store
  (lambda () '()))

;; initialize-store! : () -> Store
;; Sets the-store to an empty list (initial memory state)
(define initialize-store!
  (lambda ()
    (set! the-store (empty-store))))
```

- Reference Check

```
;; reference? : SchemeVal -> Bool
;; Checks if a value is a valid reference (integer index)
(define reference?
  (lambda (v)
    (integer? v)))
```

- Reference Allocation

```
;; newref : ExpVal -> Ref
;; Allocates a new memory slot for val, returns its index (reference)
(define newref
  (lambda (val)
    (let ((next-ref (length the-store)))
      (set! the-store
        (append the-store (list val)))
      (if (instrument-newref)
          (eopl:printf "newref: allocating location ~s with initial contents ~s~%" next-ref val)
          'no-log) ;; no-log is `else` clause. Instrument-newref is a global Boolean for error-log purpose.
      next-ref)))
```

- Dereference
 

```
;; deref : Ref -> ExpVal
;; Returns the value stored at reference index
(define deref
  (lambda (ref)
    (list-ref the-store ref)))
```
- Set Reference
 

```
;; setref! : Ref × ExpVal -> Unspecified
;; Updates the value at the given reference
(define setref!
  (lambda (ref val)
    (set! the-store
      (letrec
        ((setref-inner
          ;; Returns a copy of store with val inserted at ref
          (lambda (store1 ref1)
            (cond
              ((null? store1)
               (report-invalid-reference ref the-store))
              ((zero? ref1)
               (cons val (cdr store1)))
              (else
               (cons (car store1)
                     (setref-inner (cdr store1) (- ref1 1)))))))
          (setref-inner the-store ref))))))
```
- Invalid Reference Handler
 

```
;; report-invalid-reference : Ref × Sto -> Error
;; Called when accessing out-of-bounds reference
(define report-invalid-reference
  (lambda (ref the-store)
    (eopl:error 'setref
      "illegal reference ~s in store ~s"
      ref the-store)))
```

## IMPLICIT-REFS LANGUAGE 4.3

בשפה זו, כל משתנה אוטומטית נהפך למצביע למבנה ה-Store, שם נמצע ערכו.  
אנחנו לא נשתמש יותר ב-deref, setref, newref. במקום זאת, ביצוע ההשמה עצמה ישתנה.  
השינויים עצמם באים לידי ביטוי בצורה הבאה:

### 1. Grammar Extension (lang.scm)

```
(expression  
  ("set" identifier "=" expression)  
  assign-exp)
```

### 2. Interpreter Logic (interp.scm)

- New case in value-of:  
;; Evaluates exp1, sets the reference bound to var, and returns dummy value

```
(assign-exp (var exp1)  
  (begin  
    (setref!  
      (apply-env env var)      ; look up reference of var  
      (value-of exp1 env))    ; evaluate new value  
      (num-val 27)))          ; dummy return value
```

Notes:

- apply-env now returns a **reference**, not a value.
- setref! is used to change the store at the reference.
- num-val 27 is a placeholder result, like void.

### 3. Environment Representation (data-structures.scm)

- Modified extend-env constructor:  

```
(define-datatype environment environment?  
  (empty-env)  
  (extend-env  
    (bvar symbol?)  
    (bval reference?)      ;; stores a reference, not a value  
    (saved-env environment?))  
  (extend-env-rec*  
    (proc-names (list-of symbol?))  
    (b-vars (list-of symbol?))  
    (proc-bodies (list-of expression?))  
    (saved-env environment?)))
```

Change:

bval now stores a reference (integer index to store).

Variable bindings like  $x = 5$  mean:

Allocate space in the store for 5

Store the reference (e.g., 0) in the environment under x

#### 4. Environment Lookup (environments.scm)

New apply-env:

```
(define apply-env
  (lambda (env search-var)
    (cases environment env
      (empty-env ()
        (eopl:error 'apply-env "No binding for ~s" search-var))

      (extend-env (bvar bval saved-env)
        ;; If variable found, return reference (not value!)
        (if (eqv? search-var bvar)
            bval
            (apply-env saved-env search-var)))

      (extend-env-rec* (p-names b-vars p-bodies saved-env)
        (let ((n (location search-var p-names)))
          (if n
              ;; dynamically generate a new closure in the store
              (newref
               (proc-val
                (procedure
                 (list-ref b-vars n)
                 (list-ref p-bodies n)
                 env)))
              (apply-env saved-env search-var)))))))
```

Key Behaviors:

- Returns references for normal variable bindings.
- For recursive functions via extend-env-rec\*, dynamically constructs closures and stores them via newref.
- This avoids having to pre-allocate recursive references — but means recursion in data values is not supported.

## סיכום שפות

| Feature / Language         | let            | proc                 | letrec                          | explicit-refs                          | implicit-refs                          |
|----------------------------|----------------|----------------------|---------------------------------|----------------------------------------|----------------------------------------|
| Based on                   | —              | let                  | proc                            | letrec                                 | letrec                                 |
| Functions                  | ✗ No           | ✓ Anonymous via proc | ✓ Recursive via letrec          | ✓ Recursive via letrec                 | ✓ Recursive via extend-env-rec*        |
| Recursion Support          | ✗              | ✗                    | ✓ Supports recursion            | ✓ Supports recursion                   | ✗ No recursive data (only functions)   |
| Variable Bindings          | Value          | Value                | Value                           | Value or Ref (manually created)        | Reference (automatically created)      |
| Store Usage                | ✗              | ✗                    | ✗                               | ✓ Manual store via newref              | ✓ Automatic for all variables          |
| Mutable State              | ✗              | ✗                    | ✗                               | ✓ Via setref                           | ✓ Via set                              |
| Key Expressions            | let            | proc, call           | letrec                          | newref, deref, setref, begin           | set, begin                             |
| Environment Stores         | Value directly | Closures (with env)  | Closures (with env + self-name) | Value or ref (if manually constructed) | References only                        |
| Assignments                | ✗              | ✗                    | ✗                               | ✓ Explicit: setref(x, val)             | ✓ Implicit: set x = val                |
| Dereferencing              | N/A            | N/A                  | N/A                             | ✓ Explicit: deref(x)                   | ✓ Implicit: Just use x                 |
| Closures                   | ✗              | ✓ Yes                | ✓ With recursion                | ✓ With recursion                       | ✓ With recursion                       |
| Recursive Variables (data) | ✗              | ✗                    | ✓ Via extend-env-rec            | ✓ Via extend-env-rec                   | ✗ Not possible (ref not yet allocated) |

To add a new type of value or change an existing one (e.g., support multiplication):

1. You need to update the grammar in lang.scm:  
Add a new rule for the operator (e.g., \*) if it's not already there.
2. You need to update value-of in interp.scm:  
Add a clause to evaluate the new operator using existing evaluation logic.

Example:

```
(run "[4 ! 4 ! 4 ! 4]")  
Output: (num-val 256)
```

To add a new data type of tokens (e.g., strings):

1. Update data structures:  
Define a new constructor (e.g., string-val) and an extractor (e.g., expval->str).
2. Update the lexer and grammar in lang.scm:  
Add a token to the-lexical-spec if the new type requires lexical recognition.  
Add a grammar rule to parse the new expression form (e.g., def-exp for strings).

Example:

```
Added string-val and expval->str in data structures.  
Added def-exp in the grammar for string literals.
```

To add a new operation over the new type (e.g., string concatenation):

1. Add a new derivation rule in the grammar (e.g., concat expression).
2. Add handling for the new operation in value-of.
3. Use the appropriate extractor (e.g., expval->str) to get the underlying value.

Example:

```
(run "concat (def @yosi , def @moshe)")  
Output: (string-val "@yosi@moshe")
```

## פרק 7 – שפות עם TYPE CHECKING

פרק 7 מספר הלימוד עוסק בהכנסת מערכות טיפוסים לשפות. הפרק מתאר כיצד ניתן לבחון תוכנית נתונה ולקבוע מהו הטיפוס של הערך המוחזר במידה והתוכנית תורץ. במקרה שניתן לקבוע את הטיפוס אנו אומרים שהתוכנית היא type safe. ובמקרה שלא ניתן, התוכנית נדחית עקב בעיית type error.

בפרק 7 מוצגות שתי שפות:

**שפת "ויבדוק" Checked** - היא שדרוג של שפת PROC מפרק 3. השדרוג כולל יכולת בדיקה האם תוכנית נתונה היא type safe. בשפה זו טיפוסים המשתנים הכרוכים לפרוצדורות מוצהרים ע"י המשתמש.

**שפת "ויקיש" Inferred** - שפה זו היא שדרוג של שפת Checked. בשפה זו המשתמש לא חייב להצהיר על טיפוסים המשתנים הכרוכים לפרוצדורות, והשפה מסיקה לבד, מאופן השימוש בהם, את טיפוסם.

לשם כך, נדרשת הרכבה של סדרת משוואות שפתרוןן (אם קיים) הוא טיפוס המשתנים הנעלמים. ומכך ניתן גם להסיק את הטיפוס של התוכנית כולה. הפרק מתאר אלגוריתם להסקת הטיפוס של תוכנית נתונה.

במהלך קביעת טיפוס של ערך המוחזר מתוכנית ייתכנו מספר אפשרויות לתשובה: יש טיפוס החלטי לתוכנית, יש טיפוס פולימורפי לתוכנית, או שלא ניתן לייחס לתוכנית טיפוס בגלל חוסר תאימות בין טיפוסים. המימוש של שפה זו מושאיר לכם לקריאה עצמית כחומר להעשרה. (רק המימוש הוא חומר רשות, ואילו האלגוריתם והשימוש בו לצורכי הסקת טיפוס התוכנית הוא חלק מחומר הלימוד).

### 7.1 שפת CHECKED

Adds explicit type annotations to procedure parameters.

Type checking is static.

The interpreter includes a type checker that verifies the program's correctness before execution.

Example:

$(proc ((x : int)) - (x, 1))$  ; type annotation on x

If a mismatch is found (e.g., passing a string to a procedure expecting an int), an error is raised before evaluation.

### 7.2 שפת INFERRED

Same as proc, but without explicit type annotations.

Uses a Substitution Algorithm (explained below) to infer types.

All expressions are assigned type variables, and constraints are gathered and solved.

If inference fails or is inconsistent, it reports a type error before evaluation.

Example:

$(let\ f = proc\ (x) - (x, 1)\ in\ (f\ 10))$  ; x is inferred to be int

## SUBSTITUTIONS אלגוריתם

נתאר בקצרה את שלבי האלגוריתם. ניתן לראות דוגמאות בעמוד 252-258 בספר הלימוד באנגלית.

### 1. Assign Type Variables

Each subexpression gets a fresh type variable:

$(let\ f = proc\ (x) - (x, 1)\ in\ (f\ 10))$

We label the different expressions:

| Expression                        | Label |
|-----------------------------------|-------|
| $x$                               | $t_x$ |
| $-(x, 1)$                         | $t_1$ |
| $proc\ (x) - (x, 1)\ in\ (f\ 10)$ | $t_2$ |
| $f$                               | $t_3$ |
| $(f\ 10)$                         | $t_4$ |

### 2. Generate Type Equations

We walk the AST and generate constraints for each expression (Equations):

| Expression                        | Equation                    |
|-----------------------------------|-----------------------------|
| $x$                               | $t_x = int$                 |
| $-(x, 1)$                         | $t_1 = int$                 |
| $proc\ (x) - (x, 1)\ in\ (f\ 10)$ | $t_2 = t_x \rightarrow t_1$ |
| $f$                               | $t_3 = t_2$                 |
| $(f\ 10)$                         | $t_3 = int \rightarrow t_4$ |

### 3. Calculate with substitutions – create a substitution table, and for each step do the following:

- Transfer expression to substitutions
- Check if the transfer has an effect on remaining equations or other substitutions. If it does, replace what needs to be replaced. If it doesn't affect any, continue.
- When nothing left to transfer, return final type value (if applicable).

| Equations                   | Substitutions |
|-----------------------------|---------------|
| $t_x = int$                 | empty         |
| $t_1 = int$                 |               |
| $t_2 = t_x \rightarrow t_1$ |               |
| $t_3 = t_2$                 |               |
| $t_3 = int \rightarrow t_4$ |               |



### Equations

$$t_1 = int$$

$$t_2 = int \rightarrow t_1$$

$$t_3 = t_2$$

$$t_3 = int \rightarrow t_4$$

### Substitutions

$$t_x = int$$

כך נמשיך עד שכל המשוואות יעברו לצד Substitutions וניתן יהיה להסיק שהביטוי הינו מטיפוס מסויים (או שלא...)